GEORG-AUGUST-UNIVERSITÄT GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

IndiScale Data Services

INTERNSHIP REPORT

IndiScale GmbH

Extraction of Structured Information from Unstructured Sources

---

Student:	Dimitrios Avellas
Immatriculation number:	15665725
E-Mail:	dimitrios.avellas@stud.uni-goettingen.de
Programme:	Applied Data Science M.Sc.
External Supervisor:	Dr. Alexander Schlemmer
Internal Supervisor:	Prof. Dr. Julian Kunkel
Submission date:	24.07.2026

Contents

1 Introduction **2**

2 Problem Statement **2**

3 Methods **3**

3.1 Caching as a working method 4

3.2 Reading the PDF and classifying its sections 5

3.3 Learning the vocabulary from the document 5

3.4 Retrieval 5

3.5 Tables : When to spend a vision model on them 6

3.6 Extraction in two phases 6

3.7 When two models disagree 6

3.8 Tools 7

4 Results and Evaluation **7**

5 Evaluation of the Internship **8**

6 Conclusion **9**

References **9**

A Appendix **10**

A.1 Configuration used for the reported results 10

A.2 Extraction F1 across the models tested 11

A.3 Sample output with provenance 11

1 Introduction

I completed my industry internship at IndiScale GmbH [1] in Göttingen from 15 April to 24 July 2026, about fourteen weeks at 26 hours per week, in the company's AI department. IndiScale is a small company, so this meant a handful of people rather than a division, and contact was direct. I worked day to day with my supervisor Dr. Alexander Schlemmer and with Dr. Sara Kaviani, and discussed the direction and implementation of the project with the CEO, Dr. Henrik tom Wörden.

IndiScale was founded in 2019 as a spin-off based on CaosDB, a research data management system developed at the Max Planck Institute for Dynamics and Self-Organization to manage laboratory data that are hard to represent in a conventional relational database. IndiScale maintains it commercially under the name LinkAhead: a semantic data store integrating data from instruments, electronic lab notebooks, simulations and file systems, with a data model that can be changed during operation rather than fixed at deployment.

IndiScale's open-source model is a deliberate choice, rooted in its mission to enable research. Because transparent, reproducible research requires customer sovereignty over software and data, the company has followed an open-source business model since day one.

My project had a direct ancestor. In the FDO Connect project, IndiScale had demonstrated the path from an unstructured document to a FAIR Digital Object on operational-amplifier data sheets from a battery-manufacturing use case, but that demonstrator was end-to-end LLM based and its accuracy and failure modes had never been established on a test set. My brief followed from this: design and set up a validation pipeline for comparing extraction methods, and evaluate alternatives to the LLM-only approach, in particular classical machine learning used instead of or alongside it.

I worked on the extraction step alone. The tool takes a PDF and a list of keys and returns structured JSON, with a confidence score and a provenance record for every value; it does not emit a FAIR Digital Object, a downstream step that already exists at IndiScale. Over the fourteen weeks the work outgrew the brief. What I handed over is a library, `structured-data-extraction`, published on PyPI [2]. It is domain-agnostic by construction: no field names, vocabularies or document-specific rules are checked into the code, so the same pipeline reads an operational-amplifier data sheet or an earnings report without modification. IndiScale plans to use the developed extractor in their data integration framework to enable the integration of perviously unaccessible data.

2 Problem Statement

The value of a LinkAhead instance is in what it holds: data that is semantically typed, linked and findable. Much of what customers want to load arrives as PDF. Calling those documents unstructured is a little unfair; they are heavily structured, but the structure is visual. A number in a table has a meaning because of the column above it and the row label beside it. That structure has to be reconstructed before a database can hold it.

The usual answer is a parser per document family. It works, but a parser is written against a layout; the vendor changes the template and it breaks, and every new class of document is a new development ticket. That is the manual labour research data management is supposed to remove, reappearing one level down.

The task was therefore posed without reference to a domain. Given a PDF and keys supplied at run time, return the values as structured JSON. This is not information extraction as usually posed: classical extraction assumes a fixed schema and labelled training data, and here there

is neither. The user invents the keys at run time, so the shape of a key is unknown until the document is read. A key does not always resolve to a single number: in a real data sheet it is several records, each with a value, a unit, test conditions and possibly a variant column. The schema is itself part of what has to be extracted.

The earlier demonstrator set the direction by showing what not to repeat. Its reported accuracy was 57%, over fourteen values on a single PDF, both too low to trust and too small to mean anything. But the accuracy was the lesser problem: the demonstrator emitted one value per key when there were more to be extracted. An output schema that cannot express that fact cannot be repaired by prompt tuning, which is what sent me to a rewrite rather than a repair. The measurement problem had to be solved first: a ground-truth set of annotated documents and a scorer whose number survives contact with a second document.

Three constraints shaped the rest. The pipeline had to stay domain-agnostic. It had to avoid vendor lock-in, because customer documents are often confidential and cannot be sent to a hosted model, so every LLM call must be able to run against a local model, not only a hosted commercial API. Also, the output had to be auditable: a value entering a research database and later used in an analysis must carry a record of where it came from and how much confidence it deserves. Outside scope: the tool stops at JSON, assumes born-digital PDFs, and I never tested OCR seriously. The success criterion followed from this, not a target accuracy percentage but a measurable improvement over the LLM-only baseline on more than one document.

3 Methods

The library is about 2400 lines of Python across sixteen modules, run as a chain of stages (Figure 1): a PDF is converted, embedded, and searched against the user's keys, and only the matched passages reach an LLM that discovers each key's shape and extracts into it. I will not walk through all sixteen modules; what follows are the stages where I had a real choice, with the alternative I rejected and the reason.

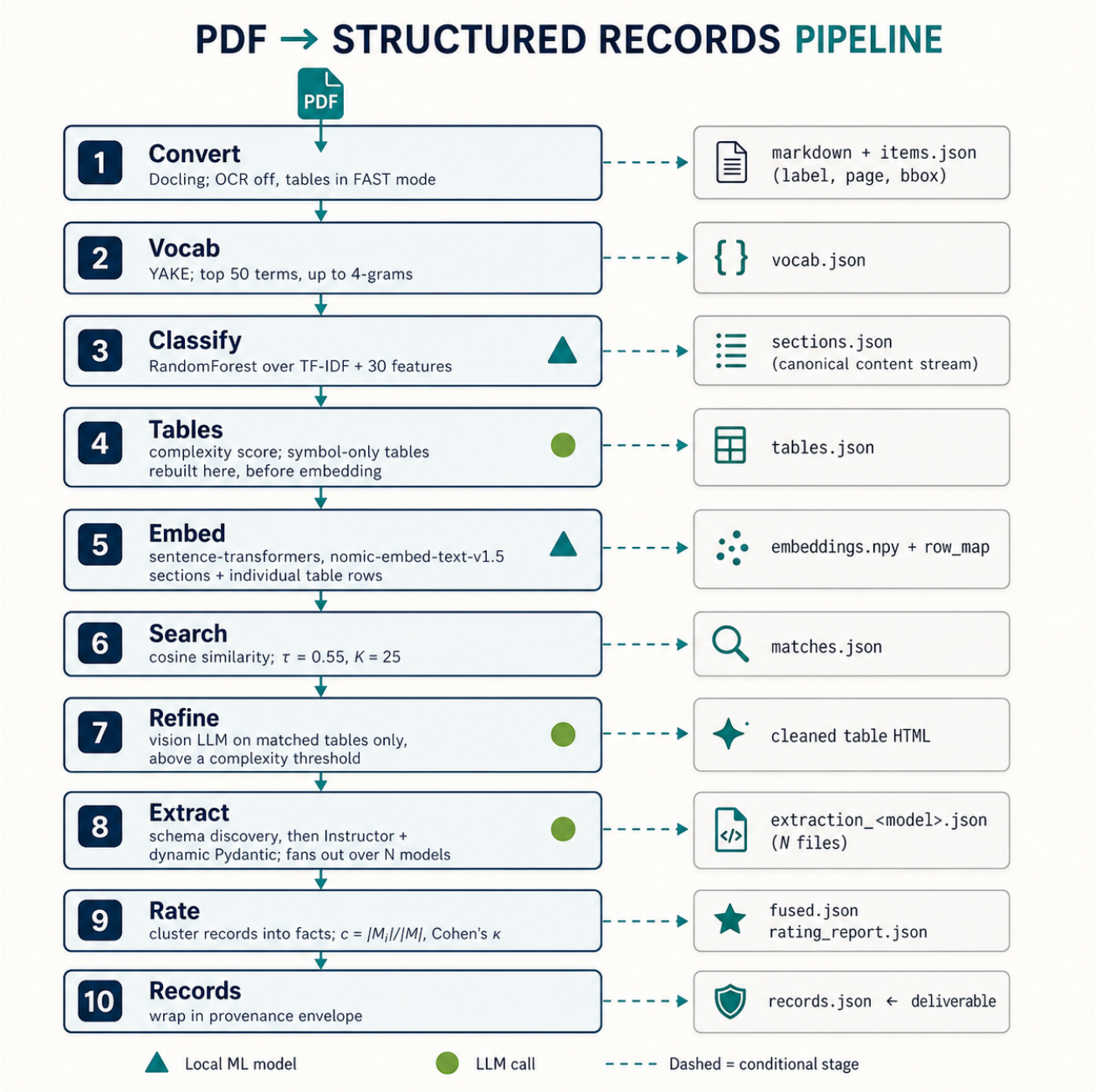


Figure 1: The extraction pipeline. Each stage reads the previous stage's artifact from disk and writes its own, so a re-run skips any stage whose output already exists. *Image generated with Claude Sonnet 4.6

3.1 Caching as a working method

Every stage writes a JSON or NumPy artifact and is skipped on a re-run if that artifact already exists. Beyond saving repeated Docling and LLM calls as I iterated on prompts, this made debugging concrete. I traced almost every bug by reading the previous stage's artifact rather than with a debugger.

An artifact is identified by the document name and the stage name, plus the model name for extraction outputs, so two extraction models over the same document never overwrite each other. One further run parameter is tracked. Keys drive everything from search onwards, so when the user's key list changes, the key-dependent artifacts are deleted and recomputed. That covers

key embeddings, matches, the discovered schema and the extractions. The document-level work is reused, since conversion, section classification and content embeddings do not depend on the keys. This is the common case in practice, a user asking a new question of a document already processed, and the expensive stages are exactly the ones that stay valid.

3.2 Reading the PDF and classifying its sections

Conversion uses Docling [3], which returns the document as a list of items, each with a label, a page and a bounding box, rather than a wall of text. OCR is off and the table model runs in `FAST` mode, since the tables that matter are rebuilt later by a vision model anyway.

Docling's labels are the weak point: about 80% accurate on operational-amplifier data sheets. This is not cosmetic: an item's type decides how it is embedded. A stack of regular expressions was the obvious repair, but each new rule broke a case the previous one handled, so I trained a RandomForest [4] to assign one of five labels: header, list, table, text and title.

The features combine TF-IDF over words and character n-grams, thirty structural and shape features, and context: neighbouring labels, distance to the nearest header, and Docling's own label, kept as a feature so the model learns when to overrule it. I labelled the training data myself, 6649 chunks from 30 documents across four genres. The model ships as an 89 MB joblib artifact whose output is the content stream every later stage reads, page and bounding box kept for provenance.

3.3 Learning the vocabulary from the document

A user who wants the gain of an amplifier types `gain`; the document says `gain error`, `gain drift`, `closed-loop gain`. A fixed dictionary of related terms would put domain knowledge back into the code, so instead the vocabulary is learned per document at run time. YAKE [5], an unsupervised keyword extractor, returns the fifty terms most characteristic of the document, and before search each user key is expanded with up to three of them. The domain knowledge sits in the document, the only place it can sit without being hardcoded.

3.4 Retrieval

String matching is inadequate here, so retrieval is semantic. Sections and table rows are embedded with a local sentence-transformer [9], `nomic-embed-text-v1.5` [10], and the keys with the same model.

The design is retrieval-augmented generation applied inside a single document. The corpus is one PDF rather than a collection, and the retrieval unit is a typed section or a single table row rather than a fixed-size window of text.

Matching is by cosine similarity: for a key with embedding $\mathbf{q}$ and a candidate section or row with embedding $\mathbf{d}$, both in $\mathbb{R}^{768}$,

$$\mathrm{sim}(\mathbf{q},\mathbf{d}) \;=\; \frac{\mathbf{q}\cdot\mathbf{d}}{\|\mathbf{q}\|\,\|\mathbf{d}\|} \;=\; \frac{\sum_{i=1}^{768} q_i d_i}{\sqrt{\sum_{i=1}^{768} q_i^2}\;\sqrt{\sum_{i=1}^{768} d_i^2}} \tag{1}$$

where q_i and d_i are the vector components. The cosine ignores magnitude, which lets a short key match a long row without penalty. A candidate is kept when its similarity clears a threshold

$\tau = 0.55$, capped at $K = 25$ per key; everything below never reaches an LLM. These two constants are the recall-against-cost dial, set by trial and error and left as environment variables, since an end-to-end F1 sweep over τ and K is the clearest piece of unfinished work.

The decision I am most pleased with is embedding table rows individually rather than whole tables. A whole table embeds as one muddy vector that matches everything weakly. Per row, each row is retrievable but meaningless on its own, since its parameter name lives in a group-header row above it. So each row is rebuilt with its header prepended, `Gain Error > G=1 | 0.01 | 0.04 %`, and now asserts a fact rather than a fragment.

3.5 Tables : When to spend a vision model on them

About three quarters of the answers in these documents live in tables (Section 4), which is why table handling took more of my time than anything else. Docling does not always recover a table cleanly. The repair renders the table's bounding box out of the PDF as a PNG with PyMuPDF and sends it to a vision-capable LLM for clean semantic HTML. That is expensive, so it runs only on a table that matched a key during search and whose complexity score, a heuristic in $[0, 1]$ from cell sparsity, ragged rows, group-header rows, column width and vertical merges, exceeds a threshold.

3.6 Extraction in two phases

Extraction is two LLM calls with different jobs. The first, driven by a Jinja template, receives the matched sections, the learned vocabulary and the user's keys, and defines each key's field structure (which sub-fields exist, what type and unit each carries, what it means) and names the document type. The second call performs the extraction and is constrained: the discovered schema is compiled at run time into a Pydantic model, one `List[RecordModel]` per key, and Instructor validates the output against it, with retries on failure.

After the evaluated release I will add a path that skips phase one. A user who already knows the shape they want can supply the schema directly, as keys with their sub-fields, types, units and the variant differentiators they care about; the pipeline compiles it into the same Pydantic model the extraction call is constrained to, without ever calling the schema model. This saves one LLM call and removes the discovery step's non-determinism, at the cost of the user authoring the schema, so discovery stays the default when the fields are not known in advance. This path ships in the first update to the released package, so the results in Section 4 come from the discovery path.

3.7 When two models disagree

Extraction can fan out across several LLMs in parallel, and reconciling their outputs means deciding when two records describe the same fact, not trivial when one model writes `TA = -40°C` and another `T_A = -40°C`. The rule is fuzzy on text and exact on numbers: two records are the same fact when every non-metadata field either matches or is absent from one of them. Numbers must be equal exactly, as floats; units are normalised for Unicode and dash variants through a small synonym table; free text is compared with `rapidfuzz`'s [6] `token_set_ratio`, which builds on the Levenshtein distance [7], and must clear 85 out of 100. Models phrase the same thing differently, so text has to be forgiving while numbers stay strict.

Matching records cluster into one fact whose consensus record takes the most-filled member's fields and carries a confidence

$$c(f) \;=\; \frac{|M_f|}{|M|} \tag{2}$$

where M is the set of configured extraction models and $M_f \subseteq M$ the subset that produced a record for fact f. With a single model every fact scores 1.0, which is honest, since a model cannot corroborate itself. Confidence describes a fact, not whether the models agree in general; for that I compute pairwise Cohen's kappa [8] over the models' presence vectors,

$$\kappa \;=\; \frac{p_o - p_e}{1 - p_e} \tag{3}$$

where p_o is observed agreement and p_e the agreement expected by chance. Subtracting p_e discounts what two models reach by guessing, so $\kappa = 1$ is perfect agreement and $\kappa = 0$ is chance. It is undefined when every label is identical, and that case is skipped.

3.8 Tools

Tool	**Role**	**Why this one**
Docling	PDF to items with label, page, bounding box	Item-level structure and geometry, not text alone; the bounding boxes are what make the vision repair possible.
YAKE	Per-document vocabulary	Unsupervised; needs no training corpus or dictionary, which the domain-agnostic requirement demands.
scikit-learn	Section classifier, Cohen's kappa	Trains in seconds on a small labelled set, ships as a joblib file, infers fast enough to run on every item.
sentence-transformers	Embedding sections, rows, keys	Runs locally, so no document text leaves the machine; task prefixes separate document and query vectors.
Instructor + Pydantic	Constrained LLM output	Validates the answer against a run-time schema and retries, so unparseable output never reaches the user.
rapidfuzz	Record equivalence	Fast enough to compare every record against every other, which fusion does.
PyMuPDF	Table bounding boxes to PNG	The only stage that needs pixels rather than text.
Jinja2	Prompt templates	Prompts can be dumped, edited and reloaded, so tuning one is not a code change.
python-dotenv	Configuration	Endpoint, keys, models and tuning constants come from `.env`, making the pipeline provider-agnostic in practice.

Table 1: Third-party components, and the reason each was chosen over the obvious alternative.

4 Results and Evaluation

I built a validation set of 32 real documents, electronic lab notebooks and electrical datasheets rather than the operational-amplifier sheets the demonstrator used, each with hand-annotated ground truth. A returned record is a true positive when it matches a ground-truth fact under the same-fact rule of Section 3.7, a false positive when it matches nothing, and a ground-truth

fact that no model returned is a false negative. From these counts I report precision, recall and their harmonic mean F1 (Table 2).

Model	TP	FP	FN	Recall	Precision	F1
gemma4:31b-cloud	256	19	15	0.945	0.931	0.938
deepseek-v4-flash:cloud	248	23	23	0.915	0.915	0.915

Table 2: Per-model extraction performance on the 32-document validation set, matched against hand-annotated ground truth.

The best single model, `gemma4:31b-cloud`, reached an F1 of 0.938; `deepseek-v4-flash:cloud` sat at 0.915. Other open-weight models were tested, but these two were the best in terms of F1-score and runtime. The demonstrator's 57% came from fourteen values on one PDF and is not directly comparable, but the brief asked for an improvement shown on more than one document, and 0.938 across thirty-two annotated documents meets it. Notably, gemma4 is small at 31 billion parameters, yet it followed the schema instructions more reliably than several larger models.

The result I did not expect concerns running several models at once. I had assumed fusion would raise accuracy; it does not. However, while evaluating different models with lower F1 score than `gemma4:31b-cloud`, their fusion produced better results than the individual models, but this fusion still wasn't better than the best model alone. Ensembling buys no measurable accuracy here, but leaves room for testing grid search and/or Tree-based methods when the available model list does not contain a systematic outperformer. What ensembling buys in this setting is the confidence signal: confidence is models-agreeing over total models (Equation 2) and is meaningless with one model, so the shipped configuration keeps a second model for the audit trail the project required, not for accuracy.

The agreement numbers show why that signal has to be measured properly. gemma4 and deepseek agree on 95.7% of facts, which reads like near-consensus, but their Cohen's kappa is only 0.433. The raw figure is inflated because both models find almost everything and so agree by default on the facts nobody misses; kappa discounts that chance agreement and leaves the moderate real agreement underneath, which is why the pipeline scores agreement with kappa (Section 3.7) rather than a percentage.

Two measurements guided the engineering. Roughly three quarters of the extracted values come from tables and only a few percent from prose, which justified the effort on table handling. Second, the section classifier reached an F1 of 0.928 over 10 whole-file cross-validation splits and 0.918 on op-amp datasheets alone, correcting the Docling labelling that Section 3.2 exists to fix.

5 Evaluation of the Internship

There was no job advertisement. I wrote to IndiScale on my own initiative and asked whether they had room for an intern. Two things convinced me once we started talking. The first was the open-source position: the tool would be released under MIT and the code public whether or not it turned out well. The second was the people, most of whom hold research degrees and several of whom are physicists, which is what I studied for my Bachelor. I no longer want to do physics, but I do still want to work on things scientists use, and a company that builds research data infrastructure offered a way to contribute to science from an angle I enjoy, without going back to the bench.

What I wanted to find out was where machine learning fits in a setting like this, and the answer surprised me. I had assumed the machine learning would be the substance and the rest, plumbing. In practice the learned components are small: a RandomForest of a few hundred lines

and an off-the-shelf embedding model do real work, but most of the difficulty was in deciding what to measure, building something that could measure it, and designing a pipeline whose failures could be traced.

Onboarding was formal and the introduction to SCRUM was one of the most valuable things I got in my time with the company, as these procedures rarely exist in academia, but are a must in the industry. I had the codebase and my supervisor's time from the first week, and we spoke as questions came up rather than on a rigid schedule. The Applied Data Science coursework that transferred was the unglamorous part, evaluation methodology, cross-validation, and the habit of building a ground-truth set before trusting a number; the modelling courses mattered less than I expected, precisely because the learned components stayed small. If I started again I would build the validation harness first, since I lost early weeks tuning a pipeline I could not yet measure. I would recommend IndiScale to the next intern, with one piece of advice: ask more questions and don't be afraid to deliver a non perfect solution. Sometimes this imperfection might be what a customer actually needs.

6 Conclusion

What I delivered is a domain-agnostic extraction library, published on PyPI, together with a validation harness that gives IndiScale a number it can defend: an F1 of 0.938 across thirty-two annotated documents, a real improvement over the LLM-only demonstrator the brief started from.

I do not claim this is the best tool for the job in general. LlamaIndex, through its LlamaExtract API, and LangChain solve the same problem, schema-driven extraction from PDFs into validated JSON, and both are larger and better-resourced than anything one intern writes in fourteen weeks. What this library has that they do not is a hard constraint on where the data goes. LlamaExtract is a hosted cloud service, and sending confidential customer documents to it is exactly what companies cannot do; every model call here can instead run against a local, open-weight model.

That freedom has a cost, and it is the honest limit of the work: the pipeline's quality depends on the models behind it. The reported numbers come from cloud-hosted open-weight models. The provider-agnostic design means nothing is locked to the cloud: for any OpenAI-compatible endpoint, switching provider is an `.env` change, not a code change. I ran the full pipeline on a local Ollama stack (`ornith:9b` for extraction, `qwen3.5:9b` for schema discovery, `glm-ocr` for vision) on a 16 GB Macbook. It needed some daemon-specific tuning to fit the hardware: Ollama's default output and context limits were too low for a full extraction until I raised them, but the pipeline itself was unchanged. With enough hardware, `gemma4:31b`, a vision model and a schema model all run locally, and the accuracy travels with them.

References

[1] `https://www.indiscale.com/`

[2] `https://pypi.org/project/structured-data-extraction/0.1.0/`

[3] Docling. `https://docling-project.github.io/docling/`

[4] scikit-learn: Machine Learning in Python. `https://scikit-learn.org`

[5] R. Campos, V. Mangaravite, A. Pasquali, A. Jatowt, A. Jorge, C. Nunes, and A. Jatowt, "YAKE! Keyword Extraction from Single Documents using Multiple Local Features," *Information Sciences*, vol. 509, pp. 257–289, 2020. `https://github.com/INESCTEC/yake`

[6] RapidFuzz: Rapid fuzzy string matching in Python, implemented in C++. https://pypi.org/project/RapidFuzz/1.5.0/

[7] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[8] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[10] nomic-embed-text-v1.5: Resizable Production Embeddings with Matryoshka Representation Learning https://huggingface.co/nomic-ai/nomic-embed-text-v1.5

[11] Ollama. https://ollama.com

A Appendix

A.1 Configuration used for the reported results

The pipeline is configured entirely through a `.env` file, which is what keeps it provider-agnostic; the defaults target a local Ollama [11] endpoint. The following configuration produced the numbers in Section 4:

```
LLM_BASE_URL=http://localhost:11434/v1
LLM_API_KEY=ollama
EXTRACTION_MODELS=gemma4:31b-cloud, deepseek-v4-flash:cloud
SCHEMA_MODEL=glm-5.1:cloud
VISION_MODEL=kimi-k2.7-code:cloud
EMBEDDING_MODEL=nomic-ai/nomic-embed-text-v1.5
EMBED_BATCH_SIZE=4
MAX_LLM_RETRIES=2
EXTRACTION_WORKERS=4
VOCAB_TOP_N=50
VOCAB_MAX_NGRAM=4
SIMILARITY_THRESHOLD=0.55
SEARCH_TOP_K=25
VISION_COMPLEXITY_THRESHOLD=0.15
VISION_PARALLELISM=4
EXPAND_KEYS=true
```

A.2 Extraction F1 across the models tested

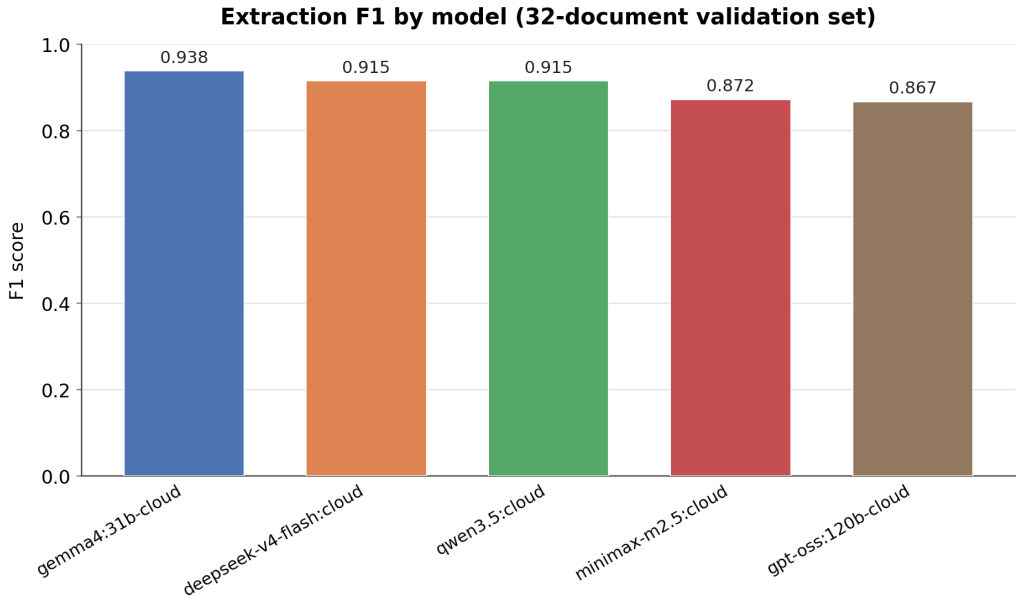


Figure 2: Extraction F1 on the 32-document validation set. The pipeline is identical across bars (same schema model, vision model, retrieval, τ, K and ground truth); only the extraction model changes. Each run is single-model, scored independently against the ground truth with the same-fact rule of Section 3.7.

A.3 Sample output with provenance


```
{
  "AMZN-Q3-2025-Earnings-Release": {
    "type": "ExtractionResult",
    "provenance": {
      "source_file": "AMZN-Q3-2025-Earnings-Release.pdf",
      "document_type": "quarterly earnings report",
      "models": {
        "extraction": ["deepseek-v4-flash:cloud", "gemma4:31b-cloud"],
        "schema_discovery": "glm-5.1:cloud",
        "vision": "kimi-k2.7-code:cloud",
        "embedding": "nomic-ai/nomic-embed-text-v1.5"
      },
      "user_keys": ["net income"]
    },
    "key_mapping": {
        "net_income": "net income"
      },
    "net_income": [
      { "confidence": 1.0, "models": ["deepseek-v4-flash:cloud",
                                      "gemma4:31b-cloud"],
        "differentiator": "Three Months Ended September 30, 2025",
        "value": 21187.0, "value_unit": "USD (millions)",
        "metadata": { "section_type": "table", "specified": true } },
      { "confidence": 1.0, "models": ["deepseek-v4-flash:cloud",
```

```
                                    "gemma4:31b-cloud"],
        "differentiator": "Three Months Ended September 30, 2024",
        "value": 15328.0, "value_unit": "USD (millions)",
        "metadata": { "section_type": "table", "specified": true } },
      { "confidence": 1.0, "models": ["deepseek-v4-flash:cloud",
                                    "gemma4:31b-cloud"],
        "differentiator": "Nine Months Ended September 30, 2025",
        "value": 56478.0, "value_unit": "USD (millions)",
        "metadata": { "section_type": "table", "specified": true } },
      { "confidence": 0.5, "models": ["deepseek-v4-flash:cloud"],
        "differentiator": "Trailing Twelve Months Ended June 30, 2025",
        "value": 70623.0, "value_unit": "USD (millions)",
        "metadata": { "section_type": "table", "specified": true } }
    ]
  }
}
```

Annex: Declaration on the Use of ChatGPT and Comparable Tools in the Context of Examinations

In this report, I have used ChatGPT or another AI tool as follows:

- ☐ not at all
- ☐ during brainstorming
- ☐ when creating the outline
- ☒ for the development of software source texts
- ☒ for optimising or restructuring software source texts
- ☒ for proofreading or optimising

I hereby declare that I have stated all uses completely. Missing or incorrect information will be considered an attempt to cheat.